REPORT Review

Member 1:

Name : Pradipta Pal
Register No : 24BRS1116
Member 2:

Name : Aniket Prasun Singh
Register No : 24BRS1123

Software Design Document

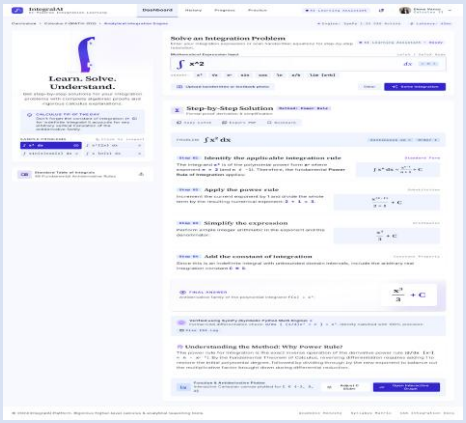
a. Design Principles We Applied

While designing IntegralAI, we followed four main software design principles: abstraction, modularity, cohesion, and low coupling. These principles helped us keep the system organized and make it easier to modify or extend later.

Abstraction

We used abstraction to hide the complex mathematical processing from the user and from other parts of the application. For example,the frontend only needs to send an integration problem to the backend using the API. It does not need to know how the formula is selected, how the integration is calculated, or how SymPy verifies the answer.

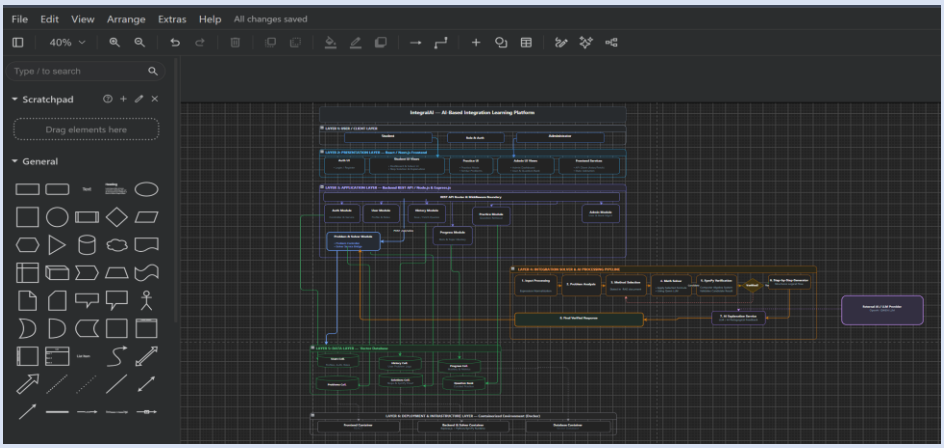
Similarly, the backend uses a **Solver Service Bridge** to communicate with the integration-solving pipeline. This keeps the actual mathematical logic hidden behind a simpler interface.



Modularity

We divided the backend into separate modules based on their responsibilities. The architecture contains modules such as **Auth, User, Problem & Solver, History, Practice, and Admin**.

For example, the History Module is responsible for saving and retrieving previous problems, while the Progress Module handles statistics and topic mastery. This makes the project easier to understand and maintain because each module has a specific purpose.

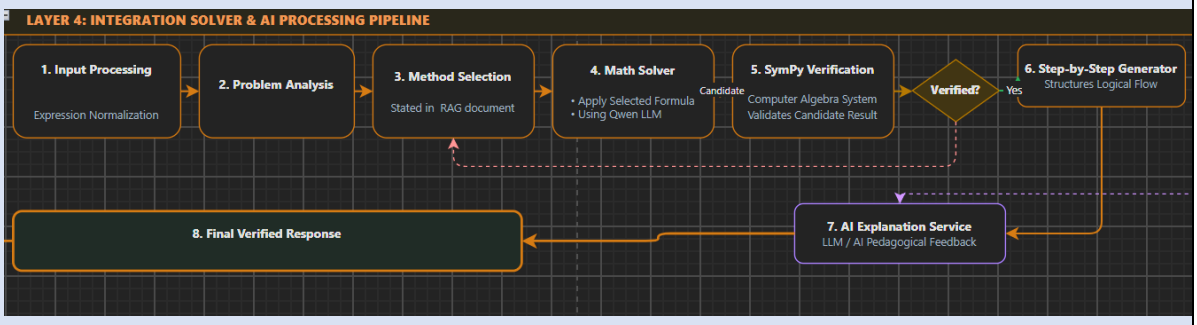


Cohesion

We tried to keep related functionality together inside the same module. For example, authentication-related operations are kept inside the **Auth Module**, while solving-related operations are handled by the **Problem & Solver Module**.

The integration pipeline also follows a clear sequence: input processing → problem analysis → method selection → mathematical solving → SymPy verification → step generation → explanation.

This gives each part of the system a focused responsibility instead of putting all the logic in one place.



Low Coupling

We reduced dependency between different parts of the system by using the **REST API** as the communication boundary between the frontend and backend. The frontend communicates with the backend through the API rather than directly accessing the database or mathematical solver.

For example:

```
const response = await axios.post("/api/solve", {
    problem: expression
});
```

This means we can change the internal solver implementation without making major changes to the frontend, as long as the API response remains compatible.

b. . High-Level Architecture

Architecture Style: Layered Architecture with Modular Services

For IntegralAI, we selected a **Layered Architecture with Modular Services**. We chose this style because our system contains several different responsibilities, such as user interaction, API handling, mathematical processing, data storage, and deployment.

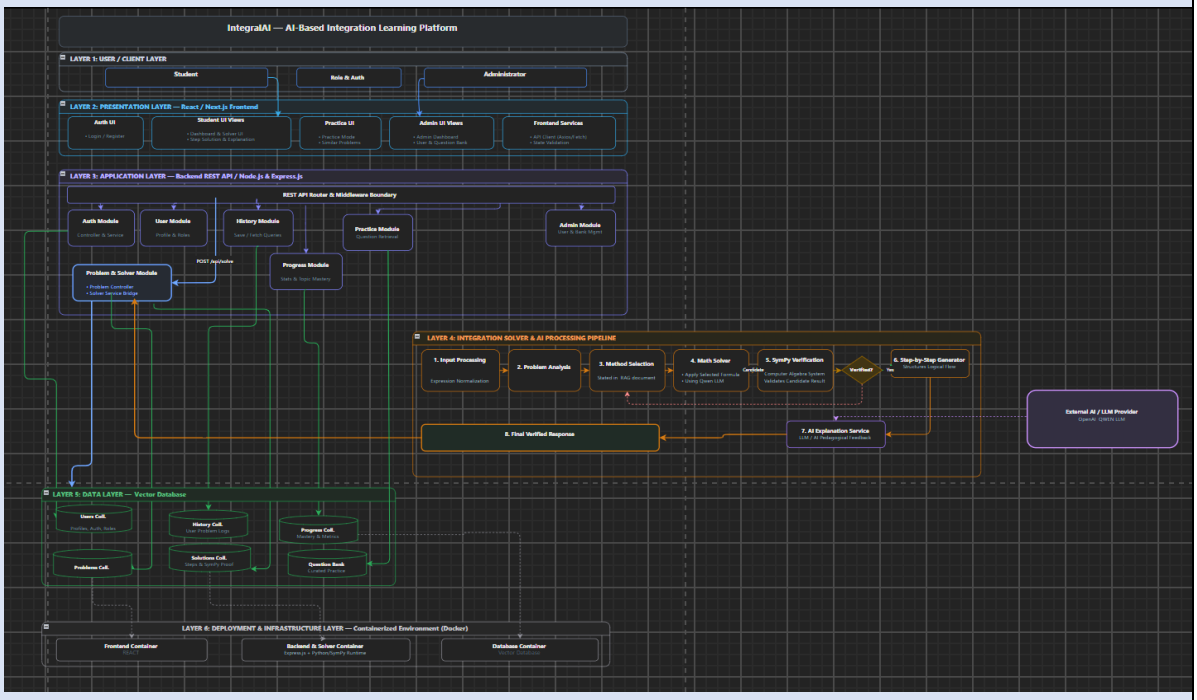
Our updated architecture is divided into six layers:

1. **User / Client Layer** – Student and Administrator
2. **Presentation Layer** – React/Next.js frontend
3. **Application Layer** – Node.js and Express.js REST API
4. **Integration Solver & AI Processing Pipeline**
5. **Data Layer** – Database and collections
6. **Deployment & Infrastructure Layer** – Docker

The main flow of the system is:

Student → Frontend → REST API → Problem & Solver Module → Integration Solver → SymPy Verification → Step-by-Step Solution → AI Explanation → Final Response → Frontend.

The architecture also separates the mathematical processing from the user interface. The solver pipeline performs expression normalization, problem analysis, method selection, formula application, verification, and explanation generation.



We selected this architecture because it gives clear separation of responsibilities. If we later change the frontend, database, or mathematical solver, the changes can mostly be made within their respective layers without affecting the complete system.

c. User Interface Design

1. Login / Register Screen

This screen provides a simple way for users to access their account. We kept the number of fields limited so that users can quickly log in or create an account.

2. Student Dashboard

The dashboard provides access to the main features such as solving problems, viewing history, checking progress, and practicing similar problems. This avoids making the user search through multiple pages to find the main functions.

3. Problem Solver Screen

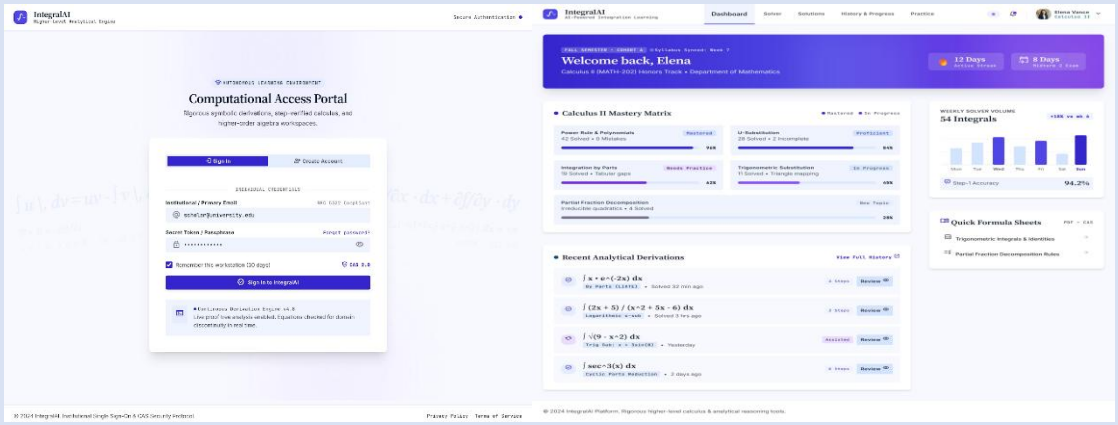
The solver screen is the main feature of IntegralAI. Users can enter their integration problem and submit it for solving. The interface is kept focused on the problem input and solve action so that the user understands what to do immediately.

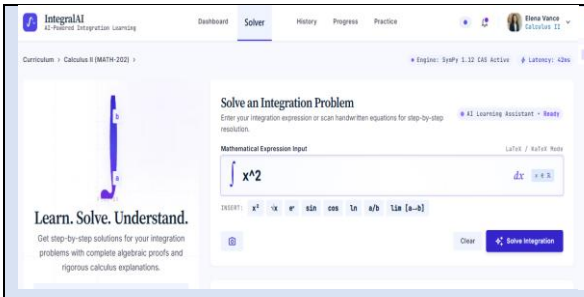
4. Step-by-Step Solution Screen

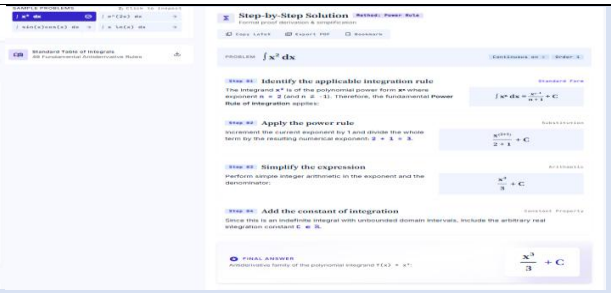
Instead of displaying only the final answer, the solution screen presents the solution in steps. This matches the main purpose of IntegralAI, which is to help students understand **how** an integration problem is solved.

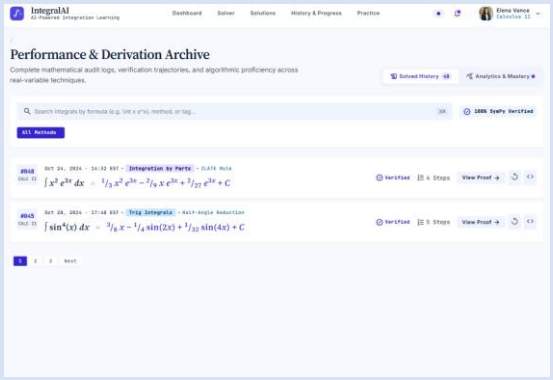
5. History / Progress Screen

The history section allows students to view previously solved problems, while the progress section provides information about their learning and topic mastery. The backend architecture also contains separate History and Progress modules for these responsibilities.









How We Made the UI User-Friendly

We focused on keeping the interface **simple and easy to understand**. We grouped related features together, used clear labels, kept the important actions visible, and avoided unnecessary elements. The main solving process is also divided into understandable stages, so students can focus on the problem, solution steps, and explanation without being overwhelmed by the technical processing happening in the backend.

d. Design Decisions

1. Layered Architecture

We chose a layered architecture to separate the frontend, backend, solver, database, and deployment responsibilities. This makes the system easier to understand and maintain.

2. Separate Mathematical Solver and AI Explanation

We separated the mathematical solving and verification process from the explanation part. The solver applies the selected formula, while **SymPy is used to verify the candidate result** and the AI service helps generate the explanation.

This gives us better control over the correctness of the mathematical answer.

3. REST API Between Frontend and Backend

We used a REST API boundary so that the frontend and backend are not directly dependent on each other's internal implementation. The architecture represents this through the REST API Router and Middleware Boundary.

4. Separate Modules for Different Features

Features such as authentication, users, solving, history, progress, practice, and administration are separated into modules. This improves organization and makes it easier to work on one feature without unnecessarily affecting others.

5. Docker-Based Deployment

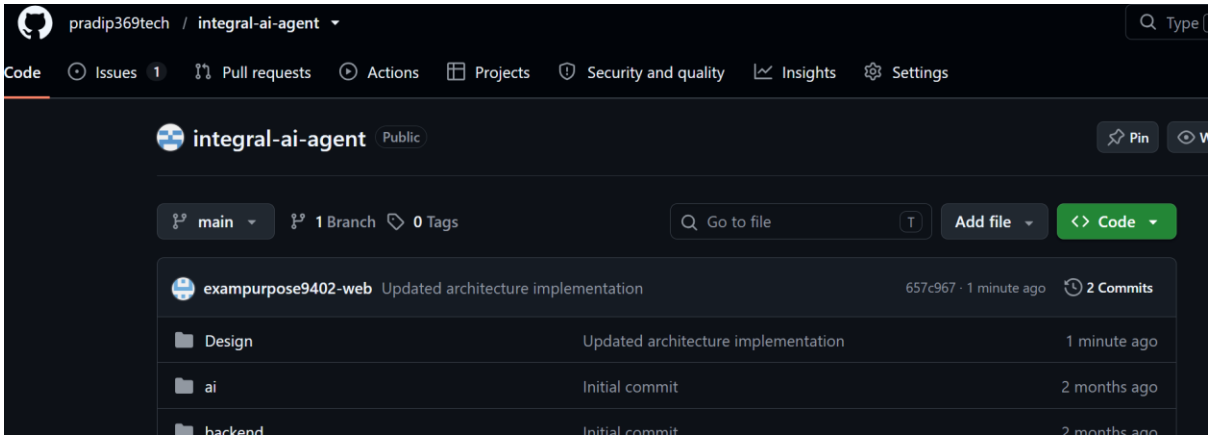
We selected Docker to keep the application environment consistent. The architecture separates the frontend, backend/solver runtime, and database into containerized environments.

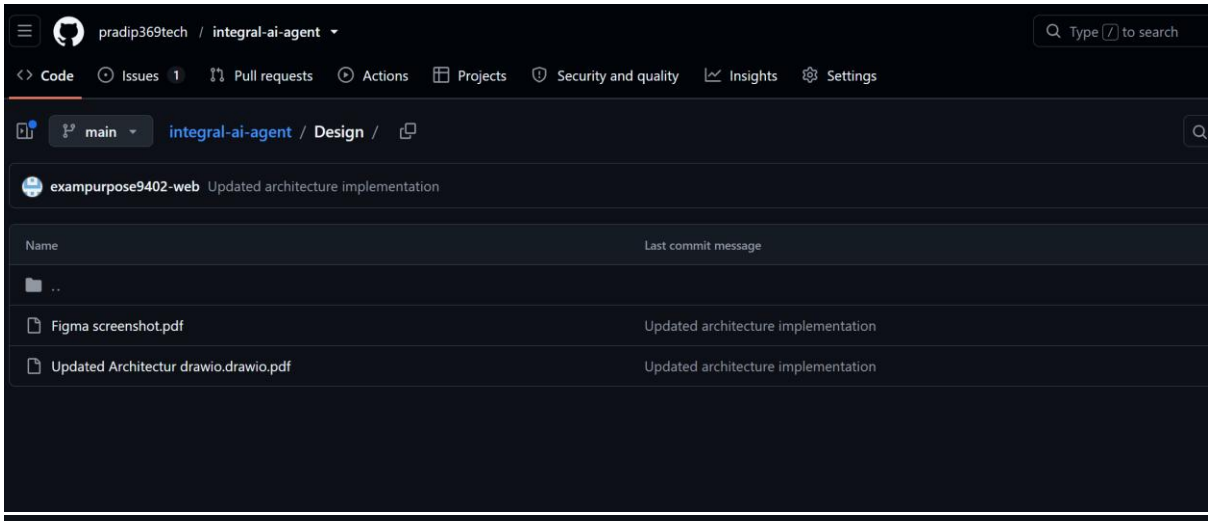
This makes the project easier to run and deploy across different environments.

GitHub Repo Updates

GIT HUB REPO LINK:

https://github.com/pradip369tech/integral-ai-agent





Topic : Integral AI

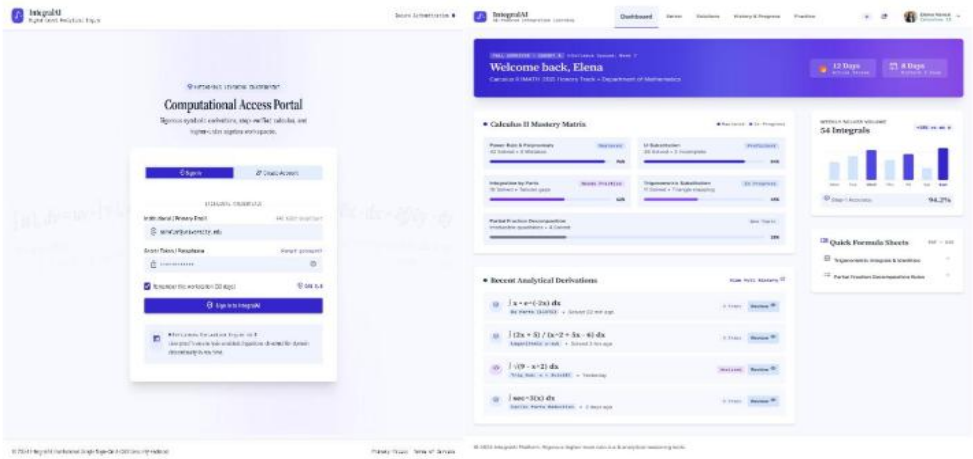
1. Software Design

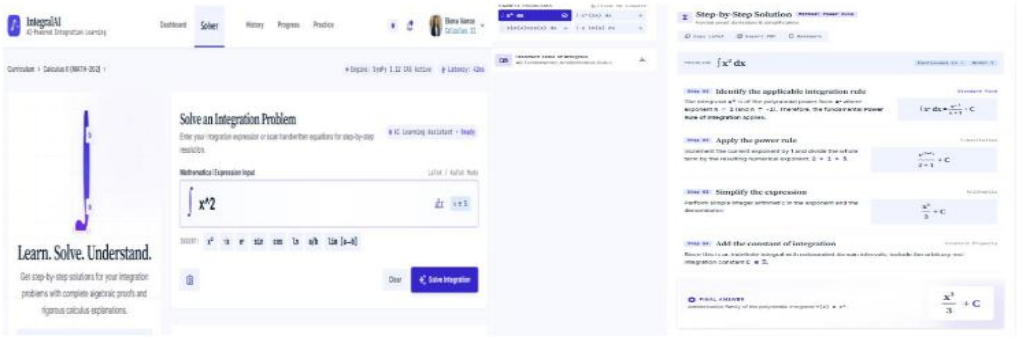
IntegralAI is an AI-based integration learning platform designed to help students solve integration problems and understand the solving process. The software uses a layered architecture that separates the user interface, backend services, integration solver, data storage, and deployment. The frontend is designed using React/Next.js, while the backend uses Node.js and Express.js to handle API requests and different modules such as authentication, solving, history, progress, practice, and administration. The integration solver processes the problem, selects the suitable method and formula, generates the solution, and uses SymPy for verification. An AI/LLM service is also used to support explanations. The system follows abstraction, modularity, cohesion, and low coupling so that each part has a clear responsibility and can be changed easily without affecting the entire system. The user interface is kept simple and student-friendly, with a clear problem input area and step-by-step solution display. Docker is used to provide a consistent deployment environment.

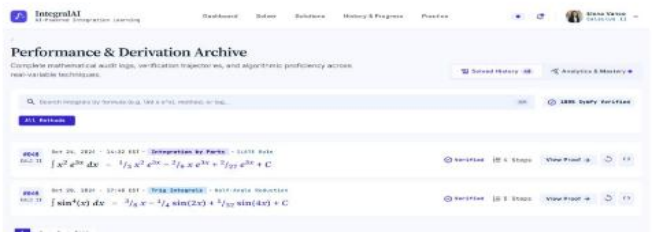
2. Project Overview

IntegralAI is an AI-powered educational web application designed to help students learn and solve calculus integration problems effectively. Users can either type an integration problem or upload an image of a handwritten or printed question. The system analyzes the problem using Artificial Intelligence, identifies the most appropriate integration technique, and generates a detailed step-by-step solution with explanations. It also stores solved problems and learning progress, allowing users to review previous work, practice similar questions, and improve their understanding of integration concepts. The platform also includes an administrator dashboard for managing users, question banks, and monitoring system performance.

3. Problem it Solves


IntegralAI
Computational Access Portal
Welcome back, Elena
Calculus II Mastery Matrix
Recent Analytical Derivations
54 Integrals
Quick Formula Sheets


IntegralAI
Dashboard
Solver
History
Progress
Practice
Solve an Integration Problem
Learn. Solve. Understand.
Get step-by-step solutions for your integration problems with complete algebraic proofs and rigorous calculus explanations.
Step-by-Step Solution
Identify the applicable integration rule
Apply the power rule
Simplify the expression
Add the constant of integration
FINAL ANSWER


IntegralAI
Performance & Derivation Archive